Manipulación de Cadenas y Punteros en C/C++: Control de Acceso por Edad Operando Directamente en Memoria RAM

Aramis Demaria

E.E.S.T. N.º 1 “Eduardo Ader” – Vicente López

Laboratorio de Programación (LPR) – 5.º Año 3.º A-B

Prof. Mansilla Muñoz York Elías

2026

Resumen

Se desarrolla un control de acceso que manipula la variable de edad mediante un puntero, evidenciando la organización de la memoria RAM en casilleros con dirección hexadecimal. Se analizan el operador de dirección (&), el de desreferenciación (*) y la diferencia entre imprimir *p* y **p*. Las pruebas con edades de 16 y 21 años muestran la bifurcación APROBADO y RECHAZADO y la dirección física variable. El informe explica por qué *cin >> p* sin asterisco corrompe la dirección y por qué el sistema operativo emplea notación hexadecimal para las direcciones, consolidando bases de bajo nivel para el Proyecto Integrador Anual.

Palabras clave: puntero, desreferenciación, dirección de memoria, RAM, hexadecimal, control de acceso

Abstract

An access control that manipulates the age variable through a pointer is developed, evidencing RAM organization into cells with hexadecimal addresses. The address-of (&) and dereference (*) operators and the difference between printing *p* and **p* are analyzed. Tests with ages 16 and 21 show the APPROVED and REJECTED branches and the variable physical address. The report explains why *cin >> p* without an asterisk corrupts the address and why the operating system uses hexadecimal notation, consolidating low-level foundations for the Annual Integrative Project.

Keywords: pointer, dereferencing, memory address, RAM, hexadecimal, access control

Introducción

Para comprender C y C++ en bajo nivel es necesario visualizar la memoria RAM como casilleros numerados con dirección única. Un puntero es la variable que almacena esa dirección; desreferenciar significa abrir el casillero.

La Actividad 4 solicita un sistema mínimo de control: leer la edad mediante puntero y decidir el acceso para mayores de 18 años, mostrando valor y dirección hexadecimal. El objetivo pedagógico es disociar dirección y contenido.

Revisión de la Literatura

La memoria RAM es el almacenamiento temporal de variables durante la ejecución. La dirección hexadecimal es la representación en base 16 con prefijo *0x*, compacta y alineada a potencias de dos. El puntero es la variable que guarda la dirección de otra variable; el operador de dirección devuelve la ubicación física y el operador de desreferenciación accede o modifica el contenido del casillero apuntado (Stroustrup, 2013; cplusplus.com, 2024).

Expresión	Qué es	Qué imprime (ejemplo)
p	Puntero (dirección)	0x61fe14
*p	Contenido apuntado	21
&edad;	Dirección de edad	0x61fe14
edad	Variable directa	21

Nota. Diferencia entre puntero, contenido y dirección.

Método

El código en *src/main.cpp* define *int edad = 0; int* p = &edad;*. La entrada se realiza con *cin >> *p*, donde el asterisco es obligatorio. La decisión se evalúa con *if (*p >= 18)* sobre el contenido, no sobre la dirección. La salida muestra **p* y *p* en hexadecimal.

La estructura */cadenaspunteros* contiene *src/*, *docs/* y *capturas/*. La compilación se ejecutó con *g++ src/main.cpp -o src/cadenaspunteros.exe*. Las capturas incluyen la ejecución con 16 y 21 años y el esquema de memoria RAM. Las pruebas contemplaron el caso límite de 18 años (aprobado) y 17 (rechazado), verificando que la dirección hexadecimal varía entre ejecuciones por la aleatorización del espacio de direcciones.

Resultados

Con 21 años el sistema informó ACCESO APROBADO, edad 21 y dirección *0x61fe14*. Con 16 años informó ACCESO RECHAZADO con la misma lógica. El programa no distingue si el dato se ingresó de forma directa o mediante puntero, dado que ambos escriben en el mismo casillero.

Figura 1. Ejecución mostrando ramas APROBADO y RECHAZADO y valor hexadecimal (capturas/ejecucion_punteros.png). Figura 2. Esquema de casilleros RAM, puntero y operadores (capturas/esquema_ram.png).

Discusión

La instrucción *cout << p* imprime la dirección hexadecimal (“dónde”), mientras que *cout << *p* imprime el valor (“qué”). Confundir ambas es el error más frecuente al iniciarse con punteros.

La expresión *cin >> p* sin asterisco intenta sobrescribir la dirección misma —un puntero no es un entero—, lo que produce error de tipos o corrupción de memoria; *cin >> *p* escribe en el entero apuntado, que sí es el tipo esperado. El sistema operativo emplea hexadecimal por compacidad y correspondencia directa con binario: un dígito hexadecimal equivale a cuatro bits y dos dígitos a un byte, lo que resulta legible para rangos amplios y alineado con tamaños de página.

Conclusión

Se demostró el uso correcto de los operadores de dirección y desreferenciación y la trazabilidad en memoria RAM. El patrón puntero y desreferenciación es base para arreglos, pasaje por referencia y estructuras dinámicas de las actividades siguientes. Como trabajo futuro se propone aplicar el mismo patrón a cadenas en C (*char**) y a estructuras con el operador flecha.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

cplusplus.com. (2024). *Pointers in C++*. https://cplusplus.com/doc/tutorial/pointers/

Microsoft. (2024). *Address-of and dereference operators*.

https://learn.microsoft.com/es-es/cpp/cpp/

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.